Misiones X500 en simulación

De los ejemplos a los retos: mundos, PX4, lidar, cámara y ArUco

Imad Jared Cabrera Trejo

Vespo UAV Drones Autónomos

11 de noviembre de 2026



Agenda



1. Panorama: qué es x500_retos
2. Preparar y correr
3. Marcos de referencia
4. px4_link.hpp: lo que comparten todas
5. Máquinas de estados
6. Cómo están hechos los mundos
7. Nivel 1: moverse bien
8. Nivel 2: decidir solo
9. Nivel 3: ver con cámara
10. Calificar, depurar y chuleta

Contenido

1. Panorama: qué es x500_retos
2. Preparar y correr
3. Marcos de referencia
4. px4_link.hpp: lo que comparten todas
5. Máquinas de estados
6. Cómo están hechos los mundos
7. Nivel 1: moverse bien
8. Nivel 2: decidir solo
9. Nivel 3: ver con cámara
10. Calificar, depurar y chuleta

Idea central

Ya saben mover un dron con ROS 2 y PX4 (`px4_examples`). Ahora toca **resolver misiones del mundo real**, cada una en su propio mundo de Gazebo, y de la más fácil a la más difícil.

Todo es **100 % simulación**: se vale chocar, perder el control, vaciar la batería... y volver a empezar.

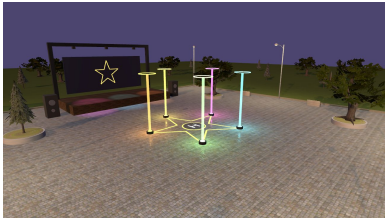


Las siete misiones

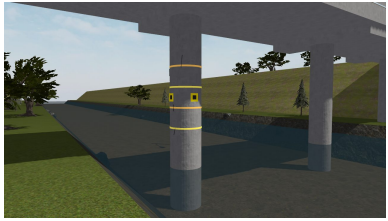
#	Misión	Mundo	Nodo	Qué se aprende
1	Estrella del show	`plaza_show`	`estrella_show`	Generar waypoints con una fórmula
2	Inspección de un pilar	`puente_rio`	`inspeccion_pilar`	Hélice con feedforward y yaw al centro
3	Entrega a domicilio	`barrio_entrega`	`entrega_domicilio`	Misión por etapas con comandos de PX4
4	Geocerca del concierto	`plaza_concierto`	`guardian_geocerca`	Un nodo que vigila y predice el frenado
5	Cruzar una cueva	`cueva`	`cruzar_cueva`	Control reactivo con lidar 2D
6	Caza del objetivo	`zona_bodegas`	`caza_objetivo`	Cámara, ArUco y seguir algo que se mueve
7	Estación de carga	`base_carga`	`estacion_carga`	Batería y aterrizaje de precisión

Nivel 3 además tiene `detector_aruco`: los "ojos" que comparten las misiones 6 y 7.

Los mundos (1/2)



`plaza_show`: la estrella dorada y las 5 balizas con aros a 6 m

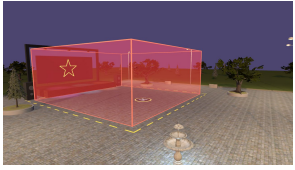


`puente_rio`: el pilar con bandas a 1, 2, 3 y 4 m



`barrio_entrega`: la plataforma naranja en el patio, a 39.4 m

Los mundos (2/2)



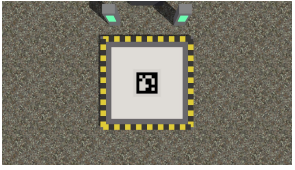
`plaza_concierto`: la geocerca roja



`cueva`: el túnel y la columna



`zona_bodegas`: la camioneta del ArUco 7



`base_carga`: la estación con el ArUco 42

Cómo está hecho un reto

Cada misión es **un nodo que ya funciona**:

- La máquina de estados, los parámetros, el log y la medición del resultado **ya están hechos**.
- Lo que pide el reto está marcado con `// TODO` y pistas.
- Mientras no lo resuelvas, el nodo vuela de forma **segura**: despega, avisa qué falta, regresa y aterriza.
- Al final, el propio nodo te dice **"Misión cumplida"** o **"Misión NO cumplida"**.

ros2 run x500_retos estrella_show

```
OFFBOARD activo, subiendo a 6.0 m
vertices() regresó una lista vacía:
falta el TODO. Aterrizando
Aterrizado. Qué tan cerca pasó...
Misión NO cumplida
```

Qué se entrega por misión

1. El nodo resuelto (los `TODO`), compilando **sin warnings**.
2. Una corrida donde el nodo diga **"Misión cumplida"** (tres corridas seguidas para la rúbrica).
3. Un README de 5 líneas: qué hace, cómo se corre, qué parámetros tiene.
4. Un video corto de Gazebo + QGroundControl.

Regla de oro: nada de números mágicos. Alturas, radios, velocidades y tiempos van como parámetros (`-ros-args -p radio:=4.0`), igual que en `orbit_mission`.

Dónde vive cada cosa

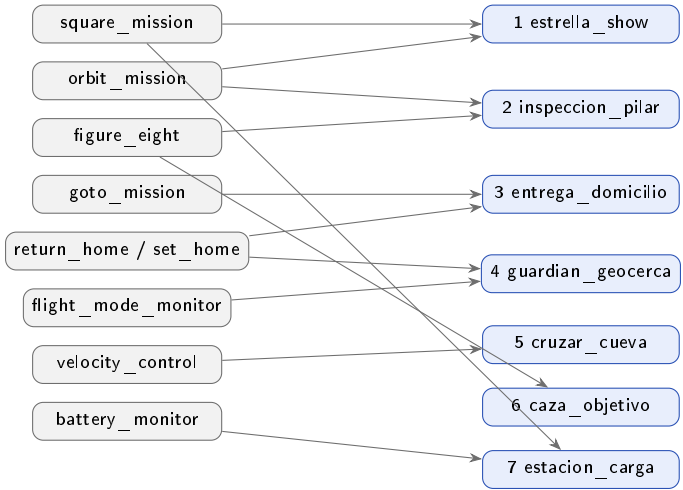


`src/x500_retos/`

- `include/x500_retos/` (px4_link, aruco_tools)
- `src/` (un .cpp por misión)
- `msg/` (Aruco, ArucoArray)
- `launch/` (cueva, caza, estacion)
- `worlds/` (los 7 mundos + spawn.yaml)
- `models/` (Fuel, texturas, x500_cam_abajo)
- `tools/` (build_worlds, make_aruco, render)
- `docs/` (capturas y esta presentación)

- `src/`: **aquí trabajan**.
- `include/`: lo que comparten todas las misiones; léanlo, no hace falta cambiarlo.
- `worlds/` se **genera** con `tools/build_worlds.py`: cambios a mano se pierden.
- `CREDITS.md`: licencias de los modelos y texturas.

De dónde viene cada misión



Contenido

1. Panorama: qué es x500_retos
2. Preparar y correr
3. Marcos de referencia
4. px4_link.hpp: lo que comparten todas
5. Máquinas de estados
6. Cómo están hechos los mundos
7. Nivel 1: moverse bien
8. Nivel 2: decidir solo
9. Nivel 3: ver con cámara
10. Calificar, depurar y chuleta

Compilar el workspace

```
cd /data/vespo_ws
vespo build                  # colcon build de todo el workspace
source install/setup.bash    # en cada terminal nueva (vespo up lo hace solo)
```

Código 1: Una vez, y cada que cambie algo en src/

- `x500_retos` instala los mundos, los modelos, los nodos, los launch y los mensajes.
- Al hacer `source`, un *hook* agrega `share/x500_retos/models` a `GZ_SIM_RESOURCE_PATH`: así Gazebo encuentra `model://house_1` y compañía.
- Si solo cambiaron su nodo: `colcon build -packages-select x500_retos`.

Levantar una misión: dos terminales

```
vespo up --sim --world plaza_show
```

Código 2: Terminal 1: agente + PX4 SITL + Gazebo + QGroundControl

```
ros2 run x500_retos estrella_show
ros2 run x500_retos estrella_show --ros-args -p points:=6 -p radius:=4.0
```

Código 3: Terminal 2: tu nodo

`-world` elige el mundo; los de este paquete aparecen junto a los de PX4 en `vespo simulate -show`. Cada mundo trae un `spawn.yaml` que dice dónde aparece el dron.

Mundo y modelo de cada misión

Misión	-world	-custom	Cómo se corre
1	plaza_show	— (x500)	ros2 run x500_retos estrella_show
2	puente_rio	—	ros2 run x500_retos inspeccion_pilar
3	barrio_entrega	—	ros2 run x500_retos entrega_domicilio
4	plaza_concierto	—	guardian_geocerca + vuelo_prueba
5	cueva	x500_lidar_2d	ros2 launch x500_retos cueva.launch.py
6	zona_bodegas	x500_cam_abajo	ros2 launch x500_retos caza.launch.py
7	base_carga	x500_cam_abajo	ros2 launch x500_retos estacion.launch.py

Ej.: `vespo up -sim -world cueva -custom x500_lidar_2d`

Los tres modelos

x500

El de siempre. Misiones 1–4.

x500_lidar_2d

De PX4: lidar 2D arriba, 1080 rayos en 270°, hasta 30 m, 30 Hz. Misión 5.

x500_cam_abajo

Nuestro: cámara hacia abajo de 960×720 a 15 Hz. Misiones 6 y 7.

Ojo

No usen `x500_mono_cam_down` de PX4 para el Nivel 3: su cámara es de 1280×960 a 30 Hz (~110 MB/s). Pasada a ROS, en una laptop de 16 GB nos llenó la RAM y trabó la computadora. Detalles en la sección de los mundos.

QGroundControl tiene que estar abierto

PX4 trae `NAV_DLL_ACT = 2`: si no hay estación de tierra, **no deja armar**.

- `vespo up -sim` ya abre QGC: no hay que hacer nada.
- Si corres `vespo simulate` solo, abre QGC con `vespo qgc`.
- Los nodos de offboard reintentan armar cada segundo: en cuanto QGC se conecta, arrancan.

consola de PX4 (pxh>)

```
WARN Preflight Fail:
No connection to the GCS
WARN Arming denied: Resolve
system health failures first
```

Parámetros sin recompilar

Todo lo que es un número de la misión se declara como parámetro de ROS 2:

```
points_ = declare_parameter("points", 5);              // picos de la estrella
radius_ = declare_parameter("radius", 5.0);            // m, del centro a un pico
altitude_ = declare_parameter("altitude", 6.0);        // m sobre el despegue
```

Código 4: Así se declaran (estrella_show)

```
ros2 run x500_retos estrella_show --ros-args -p points:=4 -p inner_ratio:=0.0
ros2 param list /estrella_show          # con el nodo corriendo
```

Código 5: Y así se cambian

El tipo sale del valor por defecto: `5` es entero, `5.0` es double. Pasar `points:=4.0` a un entero da error.

Ver qué está pasando

```
ros2 topic list | grep fmu                    # lo que publica PX4
ros2 topic echo /fmu/out/vehicle_local_position_v1 --field z
ros2 topic hz /x500/scan                      # ¿llega el lidar?
ros2 run rqt_image_view rqt_image_view /x500/aruco/debug   # lo que ve el detector
gz topic -l | grep imager                     # topics de Gazebo (no de ROS)
vespo simulate --show                         # modelos y mundos disponibles
```

Código 6: Herramientas que conviene tener a la mano

ROS 2 y Gazebo tienen topics **distintos**. El lidar y la cámara publican en Gazebo; el *launch* de cada misión levanta `ros_gz_bridge` para pasarlos a ROS.

La computadora

- **Con cargador**: Gazebo + PX4 + la GPU piden más de lo que da la batería de una laptop gamer en picos; sin cargador se nos apagó de golpe varias veces.
- **RAM**: una sesión normal usa ~1–2 GB. Si Gazebo crece sin parar (`top`, columna RES de `ruby`), algo está acumulando imágenes: ciérrenlo.
- **Tiempo real**: el factor (RTF) de Gazebo debe estar cerca de 1.0 (`gz topic -e -t /world/<mundo>/stats -n 1`). Todos los mundos corren a 1.0 en una RTX 4060 laptop.

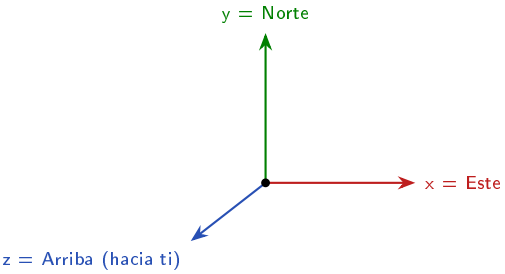
Contenido



1. Panorama: qué es x500_retos
2. Preparar y correr
3. Marcos de referencia
4. px4_link.hpp: lo que comparten todas
5. Máquinas de estados
6. Cómo están hechos los mundos
7. Nivel 1: moverse bien
8. Nivel 2: decidir solo
9. Nivel 3: ver con cámara
10. Calificar, depurar y chuleta

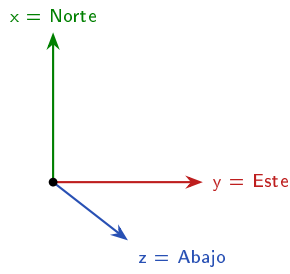
Dos maneras de decir “dónde”

Gazebo: ENU



Origen: el del mundo (lo decide quien hace el mundo).

PX4: NED



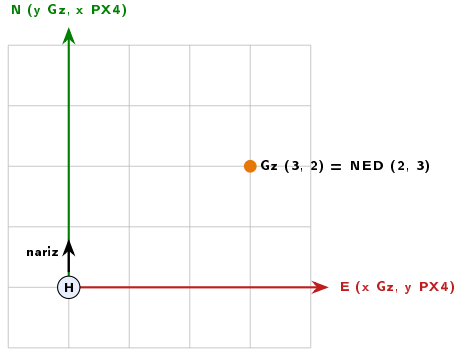
Origen: **donde arrancó PX4**, no donde está el dron.

En NED **5 m arriba es** $z = -5$. Todas las alturas que mandamos a PX4 llevan signo menos.

La convención de todos nuestros mundos

- El dron aparece en **(0, 0) de Gazebo**, mirando al **norte** (`yaw_deg`: 90 en `spawn.yaml`).
- Así el origen de PX4 coincide con el del mundo y el heading arranca en 0.
- Pasar de uno a otro es solo cambiar el orden:

$$x_{\text{PX4}} = y_{\text{Gazebo}} \quad \text{(norte)}$$
$$y_{\text{PX4}} = x_{\text{Gazebo}} \quad \text{(este)}$$
$$z_{\text{PX4}} = -z_{\text{Gazebo}} \quad \text{(abajo)}$$



Ejemplo: el pilar del puente



En `tools/build_worlds.py` el pilar está en Gazebo **(6.5, 8.0)**.

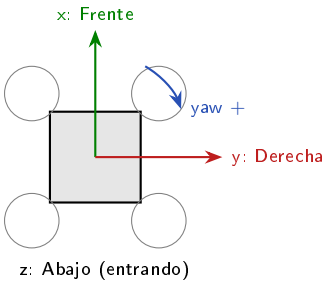
- NED norte $= y_{Gz} = 8.0$ m
- NED este $= x_{Gz} = 6.5$ m

Por eso `inspeccion_pilar` trae `center_north = 8.0` y `center_east = 6.5`.

Ojo

Si cambian de mundo o mueven el spawn, el origen de PX4 cambia y **todas** las coordenadas de la misión se recorren.

El marco del cuerpo: FRD



- PX4 describe el dron en **FRD**: x al frente, y a la derecha, z abajo.
- El **heading** (yaw) es el ángulo del frente medido desde el norte, **positivo hacia el este** (en sentido horario visto desde arriba).
- `vehicle_local_position.heading`: el yaw actual, en radianes $[-\pi, \pi]$.
- `vehicle_attitude.q`: la actitud completa (roll, pitch y yaw) como cuaternión.

Del cuerpo a NED: rotar por el heading

Si el dron va nivelado, una velocidad "adelante / derecha" se pasa a NED así:

$$\begin{pmatrix} v_N \\ v_E \end{pmatrix} = \begin{pmatrix} \cos\psi & -\sin\psi \\ \sin\psi & \cos\psi \end{pmatrix} \begin{pmatrix} v_{\text{adelante}} \\ v_{\text{derecha}} \end{pmatrix}$$

- $\psi = 0$ (mirando al norte): adelante = norte, derecha = este.
- $\psi = 90^\circ$ (mirando al este): adelante = este, derecha = sur.

Ojo

El lidar y ROS usan **FLU** (y a la *izquierda*, ángulos positivos a la izquierda). PX4 usa FRD. En la cueva hay que voltear signos: lo vemos ahí.

Ángulos: siempre envolver a $[-\pi, \pi]$

Restar ángulos "a lo bruto" falla cerca de $\pm 180°$:

$$179 - (-179) = 358 \quad \text{pero en realidad son 2.}$$

En `px4_link.hpp`:

$$\text{wrap_pi}(a) = \text{atan2}(\sin a, \cos a) \in [-\pi, \pi]$$

Úsenlo cada vez que comparen dos yaws (error de apuntado en el pilar, giro hacia un hueco, etc.).

De metros a GPS (y de vuelta)

Los comandos de PX4 (`DO_REPOSITION`, `NAV_TAKEOFF`, `DO_SET_HOME`) piden **latitud y longitud**. Para unos cuantos km basta la Tierra plana ($R = 6\,371\,000$ m):

$$\text{lat} = \text{lat}_0 + \frac{\Delta N}{R} \cdot \frac{180}{\pi} \qquad \text{lon} = \text{lon}_0 + \frac{\Delta E}{R \cos(\text{lat}_0)} \cdot \frac{180}{\pi}$$

- Los meridianos se juntan hacia los polos: por eso el $\cos(\text{lat})$ en la longitud.
- Ya están hechas: `x500::gps_offset()` y `x500::gps_distance()`.
- `lat` y `lon` van en `double` (param5 y param6 del comando): en `float` se pierde ~1 m.

¿Dónde está nuestro mundo en el planeta?

Todos los mundos usan la ubicación de los mundos de PX4 (Zúrich):

$$\text{lat}_0 = 47.397971^\circ \qquad \text{lon}_0 = 8.546164^\circ$$

Ojo

El `<magnetic_field>` del mundo corresponde a ese lugar. Si cambian la ubicación sin cambiar el campo magnético, el chequeo de brújula de PX4 falla (*"strong magnetic interference"*).

Ejemplo: la plataforma del barrio está 36 m al norte y 16 m al este de la farmacia ⇒ lat 47.3982949, lon 8.5463757.

Contenido

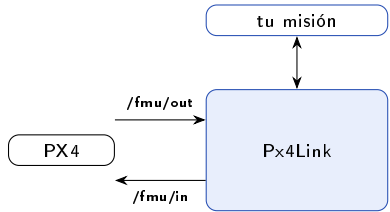
1. Panorama: qué es x500_retos
2. Preparar y correr
3. Marcos de referencia
4. px4_link.hpp: lo que comparten todas
5. Máquinas de estados
6. Cómo están hechos los mundos
7. Nivel 1: moverse bien
8. Nivel 2: decidir solo
9. Nivel 3: ver con cámara
10. Calificar, depurar y chuleta

Por qué una clase común

En `px4_examples` cada nodo repetía ~80 líneas: suscripciones, publishers, `send_command()`, `now_us()`...

Aquí eso vive en `x500::Px4Link` y cada misión se queda solo con **su máquina de estados**.

Si PX4 cambia la versión de un mensaje (`vehicle_status_v1...`), se corrige en un solo archivo.



Uso

```cpp
#include "x500_retos/px4_link.hpp"

class MiMision : public rclcpp::Node {
  x500::Px4Link px4_{this};          // crea suscripciones y publishers

  void loop() {
    const auto & l = px4_.local();  // posición NED (m), velocidad, heading
    if (px4_.armed() && px4_.in_offboard()) { ... }
    px4_.offboard(setpoint);         // heartbeat + setpoint, CADA ciclo
    px4_.command(VehicleCommand::VEHICLE_CMD_NAV_LAND);
  }
};
```

Código 7: Dentro de cualquier rclcpp::Node

Lo que lee de PX4

Método	Topic	Para qué
`status()`	`/fmu/out/vehicle_status_v1`	armado, modo (nav_state)
`local()`	`/fmu/out/vehicle_local_position_v1`	x, y, z, vx, vy, vz, heading
`global()`	`/fmu/out/vehicle_global_position`	lat, lon, alt (AMSL)
`home()`	`/fmu/out/home_position_v1`	dónde está "casa"
`attitude()`	`/fmu/out/vehicle_attitude`	cuaternión q = {w, x, y, z}

Y atajos: `armed()`, `in_offboard()`, `in_hold()`, `local_ready()`, `gps_ready()`, `distance_to(p)`.

También escucha `vehicle_command_ack` y avisa con un `WARN` cuando PX4 **rechaza** un comando (una vez por cada respuesta distinta).

El setpoint de offboard

```cpp
struct Setpoint {
  std::array<float, 3> position{kNaN, kNaN, kNaN};      // m, NED
  std::array<float, 3> velocity{kNaN, kNaN, kNaN};      // m/s
  std::array<float, 3> acceleration{kNaN, kNaN, kNaN};  // m/s^2
  float yaw = kNaN;         // rad, 0 = norte, + hacia el este
  float yawspeed = kNaN;    // rad/s, feedforward de giro
};
```

Código 8: x500::Setpoint: lo que se deja en NaN, PX4 no lo usa

`offboard(sp)` publica dos cosas, cada vez:

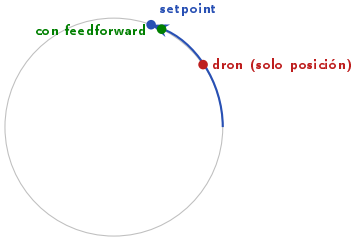
1. `OffboardControlMode`: qué campos traen valor (el *heartbeat*).
2. `TrajectorySetpoint`: los valores.

Setpoints mixtos

Basta un eje con valor para que ese tipo cuente. Así se combinan:

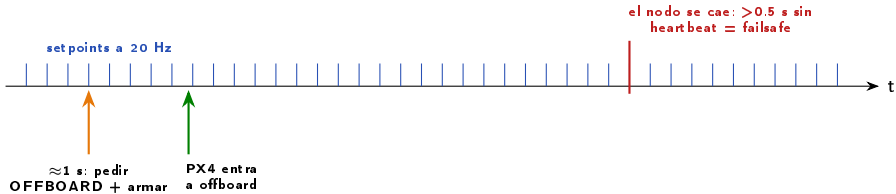
position	velocity	Qué hace PX4
$\{N, E, Z\}$	NaN	ir a ese punto
$\{N, E, Z\}$	$\{v_N, v_E, v_Z\}$	ir al punto usando la velocidad como *feedforward*
$\{\text{NaN}, \text{NaN}, Z\}$	$\{v_N, v_E, \text{NaN}\}$	avanzar por velocidad a altura fija (cueva, vuelo_prueba)
$\{N, E, \text{NaN}\}$	$\{\text{NaN}, \text{NaN}, v_Z\}$	quedarse en (N, E) bajando a v_Z (aterrizaje de precisión)

Feedforward: no ir “atrás” del setpoint



- Con **solo posición**, PX4 corrige *después* de ver el error: en una trayectoria que se mueve, siempre va atrás.
- Con la **velocidad** (y aceleración) de la trayectoria, ya sabe cómo moverse: el error queda casi en cero.
- Lo mismo con el yaw: `yawspeed` le dice qué tan rápido girar.

El heartbeat de offboard



- PX4 **no entra** a offboard si no está recibiendo setpoints ya.
- Mientras el nodo controla, hay que publicar **en cada ciclo** (todos usan un timer de 50 ms).
- Si se deja de publicar, PX4 entra en failsafe: eso también se prueba a propósito.

Comandos que usamos

Comando	Parámetros	Atajo en Px4Link
`DO_SET_MODE`	p1 = 1, p2 = 6 (OFFBOARD)	`request_offboard_and_arm()`
`DO_SET_MODE`	p1 = 1, p2 = 4, p3 = 3 (Hold)	`request_hold()`
`COMPONENT_ARM_DISARM`	p1 = 1	`arm()`
`NAV_LAND`	—	`land()`
`NAV_RETURN_TO_LAUNCH`	—	`return_home()`
`NAV_TAKEOFF`	p5, p6 = lat, lon; p7 = alt AMSL	`command(...)`
`DO_REPOSITION`	p1 = vel, p2 = 1, p5..p7 = lat, lon, alt	`command(...)`
`DO_SET_HOME`	p1 = 0, p5..p7 = lat, lon, alt	`command(...)`

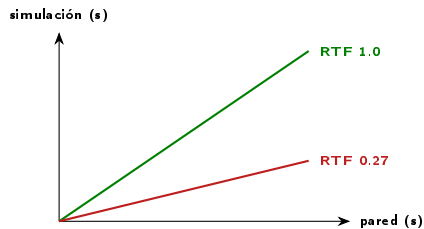
Hold antes de armar: desde Land (como queda al aterrizar) PX4 no deja armar. Por eso `entrega_domicilio` pasa por Hold antes de volver a despegar.

El reloj de PX4, no el de la pared

PX4 SITL corre en *lockstep* con Gazebo: si la simulación va lenta, PX4 también. Pero `now()` de ROS es el reloj de la pared.

En una prueba el mundo del puente corría a **0.27**×: la hélice "avanzaba" casi 4 veces más rápido de lo que el dron podía seguir y el yaw se quedaba 40° atrás.

`px4_.clock()`: el timestamp de la posición de PX4, en segundos. Úsenlo para todo lo que dependa del tiempo de la misión.



Contenido

1. Panorama: qué es x500_retos
2. Preparar y correr
3. Marcos de referencia
4. px4_link.hpp: lo que comparten todas
5. Máquinas de estados
6. Cómo están hechos los mundos
7. Nivel 1: moverse bien
8. Nivel 2: decidir solo
9. Nivel 3: ver con cámara
10. Calificar, depurar y chuleta

Todas las misiones tienen la misma forma



- **WARMUP**: ~1 s de setpoints, luego OFFBOARD + armar (reintentando cada segundo).
- **TAKEOFF**: subir sobre el despegue hasta la altura de la misión.
- **LA MISIÓN**: uno o varios estados propios (vértices, hélice, barrido...).
- **LANDING**: PX4 aterriza y desarma solo; entonces se imprime el resultado.
- **DONE**: `rclcpp::shutdown()`, así `ros2 run` regresa a la terminal.

En código: enum class + switch

```
enum class State { WARMUP, TAKEOFF, STAR, CENTER, LANDING, DONE };

void loop() {                          // timer de 50 ms
  switch (state_) {
    case State::WARMUP:
      px4_.offboard(hover(center));
      if (++counter_ % 20 == 0) px4_.request_offboard_and_arm();   // cada 1 s
      if (px4_.in_offboard()) state_ = State::TAKEOFF;
      return;
    case State::TAKEOFF: ...  break;
    case State::STAR:    ...  break;    // la misión
    case State::LANDING:
      if (!px4_.armed()) { report(); state_ = State::DONE; }
      return;
    case State::DONE: rclcpp::shutdown(); return;
  }
  if (state_ != State::LANDING && !px4_.in_offboard()) state_ = State::DONE;  // abortar
}
```

Código 9: El esqueleto (simplificado de estrella_show)

Reglas que siguen todos los nodos

1. **Cada** `case` **termina en** `break` **o** `return`. Sin él, el código sigue en el case de abajo.
2. **Una transición = un log**: `RCLCPP_INFO` al cambiar de estado, para seguir la misión en la terminal.
3. **Logs repetidos con throttle**: `RCLCPP_INFO_THROTTLE(..., 2000, ...)` como mucho cada 2 s.
4. **Si alguien toma el control, el nodo suelta**: fuera de OFFBOARD deja de mandar setpoints (el piloto o el guardián manda).
5. **El nodo se califica solo**: mide durante el vuelo y al aterrizar imprime OK / FALLÓ por criterio.

Antes de programar: dibujar la máquina

La rúbrica pide **explicar la máquina de estados en voz alta**. Antes de escribir código:

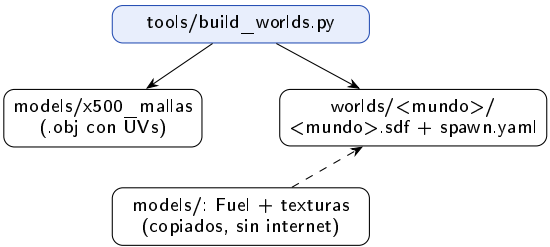
1. Escribe los estados.
2. Para cada uno: qué setpoint o comando manda.
3. Para cada flecha: qué condición la dispara.
4. ¿Qué pasa si algo falla en cada estado?

Una buena prueba: si alguien de otro equipo puede leer tu diagrama y adivinar qué va a hacer el dron, está bien.

Contenido

1. Panorama: qué es x500_retos
2. Preparar y correr
3. Marcos de referencia
4. px4_link.hpp: lo que comparten todas
5. Máquinas de estados
6. Cómo están hechos los mundos
7. Nivel 1: moverse bien
8. Nivel 2: decidir solo
9. Nivel 3: ver con cámara
10. Calificar, depurar y chuleta

Un mundo = un .sdf generado



- `python3 tools/build_worlds.py` reescribe **todos** los mundos.
- Cambiar un mundo = cambiar el script y regenerar. Cambios a mano en el `.sdf` se pierden.
- Luego `vespo build` para instalarlos.

Las piezas



Modelos de Fuel

Casas, árboles, coches, postes, fuente, bancas, camión, rocas (OpenRobotics, CC0 / CC BY). Copiados en `models/`.

Texturas

De ambientCG (CC0): pasto, adoquín, concreto, calle, roca, madera, ladrillo, grava. Color + normal + rugosidad (PBR).

Lo propio

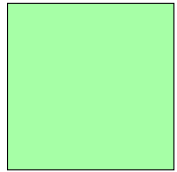
Piso, puente, farmacia, cueva, bodegas, balizas, geocerca, estación: cajas, cilindros y mallas que escribe el script.

Licencias y lista completa en `CREDITS.md`.

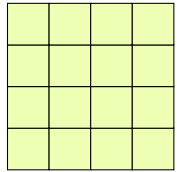
¿Por qué mallas y no un <plane>?

Un <plane> de Gazebo estira **una** imagen sobre todo su tamaño: una textura de pasto de 1 m sobre 260 m se ve borrosa.

El script escribe mallas `.obj` cuyas coordenadas UV son *metros / tamaño del mosaico*: la textura se repite cada 2–4 m.



<plane>: **1 imagen**



malla: **mosaico**

Colisiones: siempre cajas

- El motor de física de Gazebo (dartsim) no choca de forma confiable contra **mallas**.
- Por eso lo propio lleva **cajas y cilindros** como colisión, aunque se vea con malla: el puente, las paredes de la cueva (una caja cada ~1 m), las bodegas, la cerca.
- Las paredes rojas de la geocerca y los haces de las balizas son **solo visuales**: el dron las atraviesa.

Ojo

Si agregan un obstáculo, agreguen su `collision()` además del `visual()`. Si solo se ve, el dron lo atraviesa; si solo choca, el lidar o la cámara no lo ven.

Agregar algo a un mundo

```
# Un poste de 3 m en (5, -5) de Gazebo, que se ve y choca:
stage.solid(cyl(0.2, 3.0), tex('concreto_claro'), pose(5, -5, 1.5))

# Un modelo de Fuel (ya copiado en models/):
extras.append(include('oak_tree', 'arbol_extra', 12, 3))

# Una luz:
lights.append(point_light('foco', 0, 10, 3.0, (1.0, 0.8, 0.5), rng=12))
```

Código 10: Dentro de build_plaza_show(), por ejemplo

```
python3 src/x500_retos/tools/build_worlds.py
gz sdf -k src/x500_retos/worlds/plaza_show/plaza_show.sdf     # "Valid."
vespo build
```

Código 11: Regenerar, revisar e instalar

Lecciones (1/3): lo que rompía el render

1. **Texturas con el mismo nombre**: ogre2 identifica una textura por el nombre del archivo, no por la carpeta. `pasto/color.jpg` y `madera/color.jpg` salían iguales. Solución: nombres únicos (`pasto_color.jpg`).
2. **Casas negras**: los modelos viejos de Fuel usan scripts de material de Ogre 1, que Gazebo Harmonic ignora. Se cambiaron por materiales PBR con las mismas texturas.
3. **Personas caminando** (`<actor>`): congelaban el render de la cámara. No hay actores en los mundos.

Lecciones (2/3): lo que hacía lenta la simulación

El puente corría a **0.27**× tiempo real. La causa: una roca y un camión de Fuel que **no eran estáticos**. La roca pesa 2.5 t y tiene colisión de malla: la física la calculaba chocando contra el terreno a cada paso.

Con `<static>true</static>` el mundo regresó a **1.0**×.

Todo lo que es decorado va **estático**. Lo único que se mueve en nuestros mundos es el dron y la camioneta sospechosa.

Lecciones (3/3): la cámara que llenó la RAM

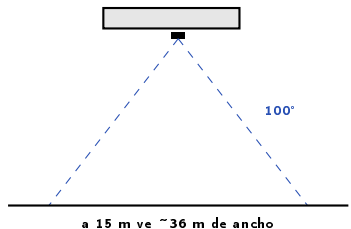
PX4 carga en todos los mundos el plugin `GstCameraSystem`: toma la primera cámara y la manda como video H.264 por UDP.

- Sin CUDA codifica por software y no alcanza.
- Su cola (`appsrc`) no tiene límite: guarda **todas** las imágenes.
- Gazebo llegó a **13.4 GB** y la computadora se trabó.

Los mundos con cámara (`zona_bodegas`, `base_carga`) declaran sus sistemas a mano, sin ese plugin (`CAMERA_WORLD_SYSTEMS`). Con eso Gazebo se queda en ~820 MB.

El modelo x500_cam_abajo

- El x500 de PX4 + una cámara hacia abajo, 10 cm bajo el centro.
- 960×720 a 15 Hz (~31 MB/s), campo de visión de 1.74 rad (100°), igual que la de PX4.
- Su airframe `4501_gz_x500_cam_abajo` vive junto al modelo; `vespo simulate` lo enlaza al build de PX4.



Lo que solo existe en simulación

La camioneta sospechosa se mueve sola: el plugin `VelocityControl` de Gazebo le da velocidad constante (3 m/s) y giro constante ($\omega = v/R$), así que da vueltas a un círculo de 26.5 m de radio sin que ningún nodo le mande nada.

$$R = \frac{v}{\omega} = \frac{3}{0.1132} = 26.5 \text{ m}$$



Herramientas del paquete

```
python3 tools/build_worlds.py                # regenerar todos los mundos
/data/vespo_ws/venv/bin/python tools/make_aruco.py      # texturas ArUco
/data/vespo_ws/venv/bin/python tools/render_world.py plaza_show /tmp/fotos \
    "aerea:0,-36,20,0,0.5,1.5708:1.5"      # foto sin abrir Gazebo
```

Código 12: En src/x500_retos/tools (las dos últimas con el venv del repo, trae OpenCV)

`render_world.py` agrega cámaras al mundo, corre Gazebo sin ventana con su propio `GZ_PARTITION` (no choca con una simulación abierta) y guarda un PNG por cámara. Así se hicieron todas las capturas de esta presentación.

Contenido

1. Panorama: qué es x500_retos
2. Preparar y correr
3. Marcos de referencia
4. px4_link.hpp: lo que comparten todas
5. Máquinas de estados
6. Cómo están hechos los mundos
7. Nivel 1: moverse bien
8. Nivel 2: decidir solo
9. Nivel 3: ver con cámara
10. Calificar, depurar y chuleta

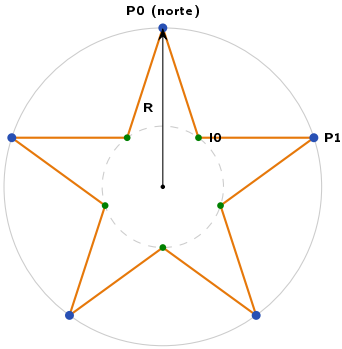
Misión 1: la estrella del show



Escenario: una empresa de espectáculos quiere una estrella de 5 picos a 6 m de altura para el show de año nuevo.

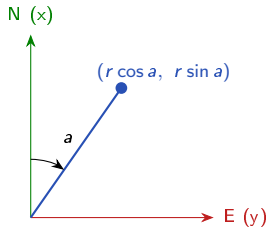
- Mundo `plaza_show`: la estrella dorada pintada en el piso.
- En cada pico, una baliza con un **aro a 6 m**: el dron tiene que pasar por adentro.
- **Viene de**: `square_mission` (waypoints en offboard).
- **Lo nuevo**: los waypoints se **generan con una fórmula**.

La geometría de la estrella



- **Picos** P_k: en el círculo de radio R, repartidos cada $360°/N$, el primero al **norte**.
- **Interiores** I_k: en el círculo de radio $R \cdot$`inner_ratio`, a medio camino (en ángulo) entre dos picos.
- Orden de vuelo: $P_0, I_0, P_1, I_1, \ldots$ y de vuelta a P_0.
- `inner_ratio = 0.382`: estrella regular de 5. `inner_ratio = 0`: polígono. Con `points:=4` debe salir **un cuadrado**.

Un punto en polares, en NED



Con el ángulo a medido **desde el norte**, **hacia el este**:

$$\text{norte} = r\cos a \qquad \text{este} = r\sin a$$

Ojo

Es al revés de la trigonometría de la escuela (desde el eje x, antihorario). Aquí $a = 0$ es el norte y $a = 90°$ el este, porque así mide PX4 el yaw.

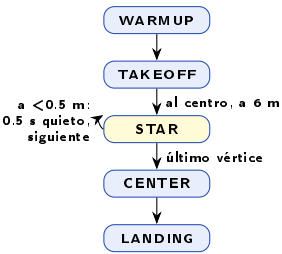
Tu parte: `vertices()`

```cpp
std::vector<Point> vertices() const {
  std::vector<Point> out;
  const float z = static_cast<float>(-altitude_);
  // TODO 1: los picos. Hay points_ picos repartidos en 360°, el primero al
  //   norte. ... Pista: el ángulo del pico k es 2*pi*k / points_ (en radianes).
  // TODO 2: los vértices interiores. Si inner_ratio_ > 0, después de cada
  //   pico va un vértice a radio radius_ * inner_ratio_, ...
  // Para agregar un punto:  out.push_back({norte, este, z});
  return out;
}
```

Código 13: Lo que trae el reto (estrella_show.cpp)

Si `vertices()` regresa la lista vacía, el nodo despega, avisa y aterriza.

Cómo vuela y cómo se califica



al aterrizar

```
Qué tan cerca pasó de cada vértice:
1: 0.15 m OK
2: 0.11 m OK
...
11: 0.05 m OK
Misión cumplida
```

Éxito: pasa a menos de `tolerance` (0.5 m) de cada vértice, regresa al centro y aterriza.

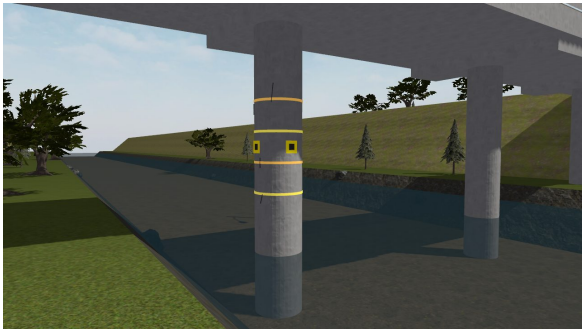
Errores típicos en la misión 1



- **Radianes vs grados**: `std::cos` recibe radianes. $2\pi k/N$, no $360k/N$.
- **División entera**: `2 * M_PI * k / points_` está bien; `k / points_ * 2 * M_PI` da 0 (`k` y `points_` son enteros).
- **double a float**: `Point` es `std::array<float,3>`; usen `static_cast<float>()`.
- **Cerrar la figura**: el último punto debe ser otra vez el pico 0.
- **z positiva**: si el dron se va al suelo, les faltó el signo menos.

Prueben `points:=6`, `points:=4 inner_ratio:=0.0` (cuadrado) y `radius:=3.0`: un buen generador funciona para cualquier N.

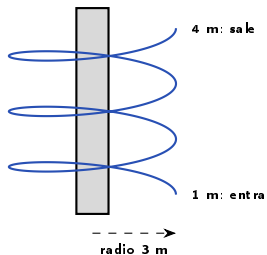
Misión 2: inspección de un pilar



Escenario: una constructora necesita fotos de un pilar de puente desde todos los ángulos y alturas.

- Subir en **hélice** alrededor del pilar: 3 vueltas de 1 m a 4 m.
- La nariz (la cámara) **siempre apuntando al pilar**.
- **Viene de**: `orbit_mission` + `figure_eight`.
- Las bandas de colores del pilar están justo a 1, 2, 3 y 4 m.

La hélice: un círculo que sube



Con velocidad angular $\omega = v/R$ y empezando en el ángulo θ_0:

$$\theta(t) = \theta_0 + \omega t$$

$$N(t) = N_c + R\cos\theta(t) \qquad E(t) = E_c + R\sin\theta(t)$$

y la altura **sube lineal** de `z_start` a `z_end` en la duración total $T = \text{vueltas} \cdot 2\pi R/v$.

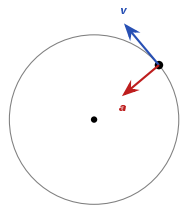
θ_0 es el punto del círculo más cercano al despegue: así llega sin rodear el pilar.

Derivar para el feedforward

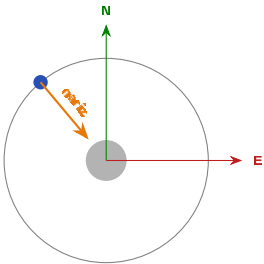
`figure_eight` ya lo hace con su curva: la velocidad es la derivada de la posición. Para un círculo:

$$\frac{d}{dt}\left(R\cos\theta(t)\right) = -R\,\omega\sin\theta(t) \qquad \frac{d}{dt}\left(R\sin\theta(t)\right) = R\,\omega\cos\theta(t)$$

- La velocidad es **tangente** al círculo, de magnitud $R\omega = v$.
- La aceleración (segunda derivada) apunta **al centro**, de magnitud $R\omega^2$.
- Y en z: si sube lineal, ¿cuánto vale dz/dt? (cuidado: NED, arriba es $-z$).



Apuntar al pilar



El yaw que apunta del dron al centro es el ángulo del vector *dron* $\rightarrow$ *centro*:

$$\psi = \text{atan2}(E_c - E,\ N_c - N)$$

(`atan2(este, norte)`: el orden importa).

Y `yawspeed` = qué tan rápido gira esa dirección: en un círculo, lo mismo que ω.

Cómo se califica la hélice

Mientras vuela, el nodo mide contra la **posición real** (no contra el setpoint):

- **Radio**: distancia real al eje del pilar, dentro de 3 ± 0.3 m.
- **Yaw**: la nariz contra la dirección real al pilar, error $< 10°$.
- **Vueltas**: suma del ángulo recorrido alrededor del pilar $\geq$ 90 % de 3.
- **Altura**: llegó a `z_end` (4 m) -0.3.

al aterrizar

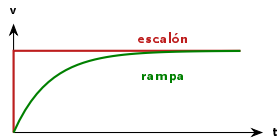
```
vueltas: 2.99 de 3.00 OK
altura máxima: 3.99 m OK
error de radio: 0.11 m OK
error de yaw: 6.1° OK
Misión cumplida
```

El extra: arrancar suave

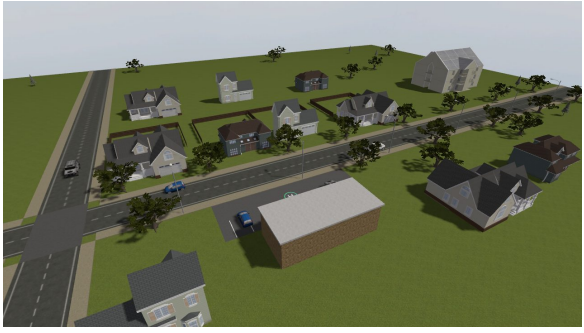
Si todo pasa menos el yaw, que se pasa de 10° **justo al empezar**:

- En $t = 0$ la velocidad del setpoint salta de 0 a v en un instante.
- El dron no puede acelerar infinito: se queda atrás un momento.
- El setpoint ya va adelante y la nariz apunta a donde *debería* estar, no a donde está.

Pista: que la velocidad suba de 0 a v en 1–2 s (y no olviden sumar ese tiempo a la duración).



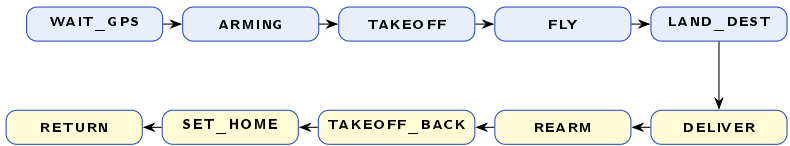
Misión 3: entrega a domicilio



Escenario: llevar un medicamento de la farmacia a una casa a ~40 m, dejarlo y volver solo.

- **Sin offboard**: PX4 vuela cada tramo con sus modos; el nodo solo decide qué sigue.
- Si el nodo se muere, el dron se queda en Hold (no hay failsafe de offboard).
- **Viene de**: `goto_mission` + `return_home` + `set_home`.

Las fases (el log las numera [1/9] . . . [9/9])



Ya funciona: de la farmacia hasta aterrizar en el patio (azul).

Tu parte (amarillo): esperar desarmado, volver a armar y despegar, **arreglar casa** y regresar con Return.

Las ayudas que ya están hechas

```
bool arm_when_ready();          // Hold y armar, reintentando cada 1 s; true si ya armó
void take_off();                // NAV_TAKEOFF sobre donde está, a cruise_alt
void go_to_target();            // DO_REPOSITION al patio: vuela y queda en Hold
void set_home_to_pharmacy();    // DO_SET_HOME con las coordenadas de la farmacia
bool at_cruise_alt() const;     // en Hold y a la altura de crucero
double home_offset() const;     // qué tan lejos de la farmacia está "casa"
```

Código 14: entrega_domicilio: tu trabajo es decidir CUÁNDO llamar cada una

Y `px4_.home().update_count`: sube cada vez que PX4 mueve "casa" (así sabes que aceptó el `DO_SET_HOME`, como en `set_home.cpp`).

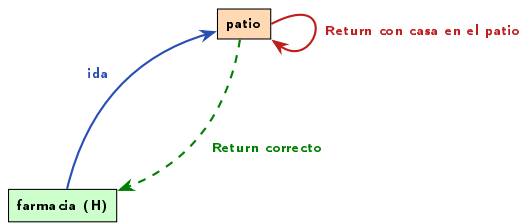
La trampa: ¿dónde quedó casa?

PX4 fija "casa" **donde arma**.

1. Arma en la farmacia ⇒ casa = farmacia.
2. Aterriza en el patio y desarma.
3. **Vuelve a armar en el patio** ⇒ casa = ...

Si mandan Return así, ¿a dónde regresa?

El nodo guarda las coordenadas de la farmacia al principio (`pharmacy_lat_`, `pharmacy_lon_`). `px4_.home()` les dice dónde cree PX4 que está casa.



Alturas y GPS en la entrega

- La plataforma está en **lat 47.3982949**, **lon 8.5463757** (36 m N, 16 m E de la farmacia).
- La ruta directa pasa **sobre la casa destino**, de ~9 m: `cruise_alt` tiene que librarla (15 m por defecto).
- Los comandos piden altura **AMSL** (sobre el nivel del mar): el nodo guarda la altura del suelo al principio y le suma `cruise_alt`.
- `Return` sube primero a `RTL_RETURN_ALT` (30 m en el x500 de SITL), vuela a casa y baja.

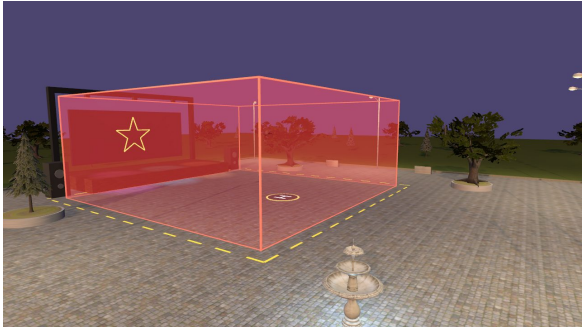
al aterrizar en la farmacia

```
De vuelta en la farmacia y desarmado. Misión de 119 s
a 0.01 m de la H OK
Misión cumplida
```

Contenido

1. Panorama: qué es x500_retos
2. Preparar y correr
3. Marcos de referencia
4. px4_link.hpp: lo que comparten todas
5. Máquinas de estados
6. Cómo están hechos los mundos
7. Nivel 1: moverse bien
8. Nivel 2: decidir solo
9. Nivel 3: ver con cámara
10. Calificar, depurar y chuleta

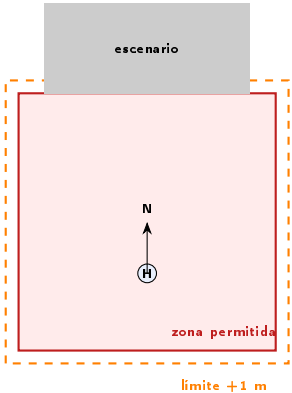
Misión 4: la geocerca del concierto



Escenario: en un concierto el dron no puede salirse de una caja de $20 \times 20 \times 8$ m.

- Un nodo **guardián** que **no vuela**: solo vigila.
- Si el dron va a salirse: **Hold**, y ya quieto, **Return**.
- Funciona **sin importar quién vuela**: un nodo, QGC o un piloto.
- **Viene de**: `flight_mode_monitor` + `return_home`.

La caja



En NED desde el despegue (parámetros del nodo):

norte	$-6 \ldots 14$ m
este	$-10 \ldots 10$ m
altura	hasta 8 m

Las paredes rojas son solo visuales: el dron las atraviesa. La **línea naranja punteada** del piso está 1 m afuera: es el criterio de éxito.

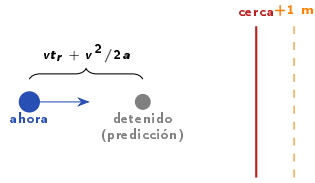
¿Por qué no basta con revisar si ya se salió?

El dron **no frena en seco**. Desde que el guardián detecta hasta que el dron se detiene:

$$d = \underbrace{v\,t_r}_{\text{reacción}} + \underbrace{\frac{v^2}{2a}}_{\text{frenado}}$$

Medido en SITL: a 4 m/s el x500 en Hold recorre todavía ~4.5 m (frena a ~2.4 m/s^2).

Hay que **predecir** dónde se va a detener y actuar si *ese* punto queda fuera.



Prueba de la fórmula

Frenando con aceleración constante a desde v:

$$v(t) = v - at \;\Rightarrow\; t_{\text{alto}} = \frac{v}{a} \qquad d = v\,t_{\text{alto}} - \tfrac{1}{2}\,a\,t_{\text{alto}}^2 = \frac{v^2}{a} - \frac{v^2}{2a} = \frac{v^2}{2a}$$

v (m/s)	d con $a = 2$ m/s^2, $t_r = 0.5$ s	lo que hay que anticipar
2	$1.0 + 1.0 = 2.0$ m	poco
4	$2.0 + 4.0 = 6.0$ m	más que la tolerancia
6	$3.0 + 9.0 = 12.0$ m	más de la mitad de la caja

Por eso `brake_accel` $= 2.0$ y `reaction` $= 0.5$ por defecto: un poco más conservador que lo medido.

Tu parte en el guardián

```
Vec3 stop_point() const {           // TODO 1: dónde se detendría
  // cada eje avanza  v * reaction_  +  v * |v| / (2 a)   (v*|v| conserva el signo)
}
case State::WATCH:                  // TODO 2: si stop_point() queda afuera
  // (outside_by(...) > 0): px4_.request_hold(), avisar y pasar a BRAKING
case State::BRAKING:                // TODO 3: casi quieto (speed() < 0.3) y en Hold:
  // px4_.return_home() y RETURNING. Si en 1 s no entró a Hold, pedirlo otra vez.
```

Código 15: guardian_geocerca: los tres TODO

Ya hechos: `outside_by(p)` (cuánto está afuera un punto), `speed()`, `distance_to_edge()` y la medición de cuánto se salió.

Probarlo: el piloto travieso

```
vespo up --sim --world plaza_concierto
ros2 run x500_retos guardian_geocerca
ros2 run x500_retos vuelo_prueba --ros-args -p direction_deg:=0.0 -p speed:=6.0
```

Código 16: Tres terminales

`vuelo_prueba` (ya completo) despega y se lanza en línea recta sin frenar, hasta que alguien le quite el control o llegue a `max_distance` (30 m, red de seguridad).

`direction_deg`: 0 = norte (hay 14 m para acelerar), 90 = este, 180 = sur.

guardian_geocerca

```
¡Se va a salir! (a 5.1 m de la
orilla, va a 5.5 m/s): Hold
Detenido a 0.00 m fuera. Return
Lo más que se salió: 0.00 m OK
```

Un detalle de la vida real: Return se sale por arriba

Return sube primero a `RTL_RETURN_ALT` (30 m en el x500 de SITL, recortado por un cono cerca de casa). En la prueba subió a 9 m: **arriba de la caja**.

El guardián no lo cuenta como falla suya (es configuración de PX4), pero avisa:

guardian_geocerca

```
Return subió a 9 m, arriba de la cerca:
baja RTL_RETURN_ALT en QGC
```

En un concierto real eso se configura antes de volar: QGC → Parámetros → Return Mode, por debajo del techo de la geocerca.

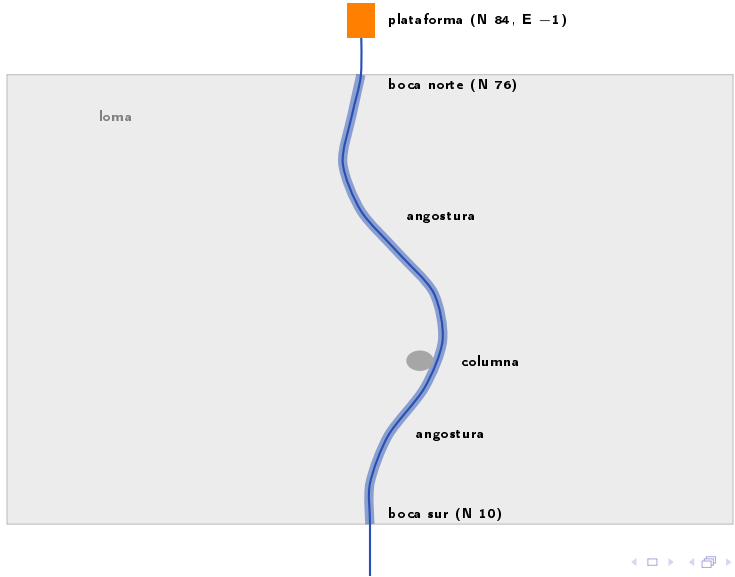
Misión 5: cruzar una cueva



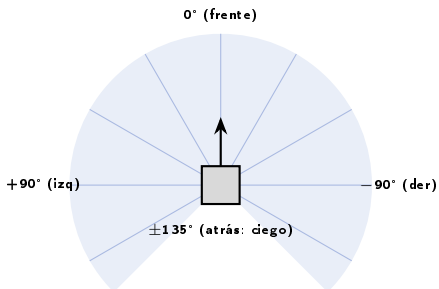
Escenario: atravesar un túnel curvo y oscuro usando **solo el lidar** para no chocar.

- Túnel de ~66 m, 5 m de ancho, 4 m de alto.
- Dos angosturas de 3 m.
- Una columna de roca a medio camino: **solo se pasa por la izquierda**.
- **Viene de**: `velocity_control`.

El mapa de la cueva



El lidar 2D: sensor_msgs/LaserScan



- 1080 rayos de `angle_min` $= -135°$ a $+135°$, cada `angle_increment`.
- `ranges[i]`: distancia del rayo i (m). **inf** si no pega en nada (hasta 30 m).
- Ángulo del rayo: `angle_min + i * angle_increment`.
- Ángulos **positivos a la izquierda** (FLU, convención de ROS).
- Va arriba del dron: el plano de escaneo está ~0.3 m sobre el cuerpo.

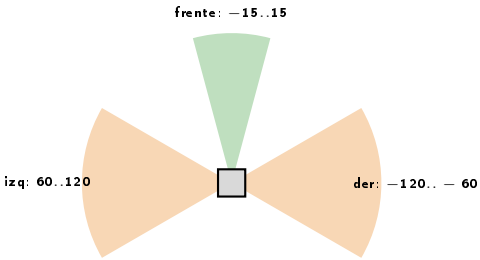
De Gazebo a ROS: el puente

```
ros2 run ros_gz_bridge parameter_bridge \
  /world/cueva/model/x500_lidar_2d_0/link/link/sensor/lidar_2d_v2/scan@sensor_msgs/msg/LaserScan[gz.msgs.LaserScan
#  ...y lo renombra a /x500/scan
```

Código 17: Lo que hace cueva.launch.py

- `[` = solo de Gazebo hacia ROS. (`]` sería de ROS a Gazebo, `@` en ambos sentidos.)
- El nombre del topic de Gazebo incluye el mundo y el modelo (`x500_lidar_2d_0`).
- `ros2 launch x500_retos cueva.launch.py` levanta el puente y tu nodo juntos.

La herramienta: el mínimo de un sector



Ya hecha en el nodo:

```
sector_min(from_deg, to_deg)
```

La distancia **más corta** que ve el lidar entre dos ángulos (positivo = izquierda). Lo que no pega en nada cuenta como `range_max`.

En el dibujo los sectores están girados: el frente va hacia arriba.

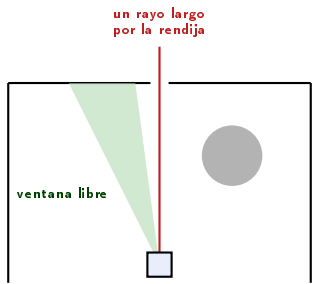
Tu parte: `steer()`, del scan a una velocidad

Se llama 20 veces por segundo. Regresa `forward`, `left` (m/s) y `yawrate` (rad/s):

1. **Centrarse**: compara izquierda contra derecha y muévete hacia el lado con más espacio, proporcional a la diferencia y con tope. Afuera de la cueva un lado da 30 m: usa `std::min(d, 3.0)` para que solo cuenten las paredes cercanas.
2. **Girar con la cueva**: busca el ángulo entre $-75°$ y $75°$ con más espacio libre y gira hacia él. Mejor: el espacio de cada ángulo como el **mínimo de una ventana** de $\pm 10°$, para no meterte por una rendija donde no cabes.
3. **Avanzar**: más rápido si el frente está libre, más lento si hay que girar mucho.

Ya hecho: si hay algo a menos de `stop_distance` al frente, no avanza; si pasa `stuck_time` sin moverse, aterriza "atorado".

Buscar el hueco: ventanas, no rayos



Un solo rayo puede “ver” muy lejos por una rendija de 20 cm donde el x500 (~0.75 m de punta a punta) no cabe.

Tomar el **mínimo** de una ventana de $\pm 10^\circ$ es pedir que *todo* ese cono esté libre.

FLU contra NED: los signos

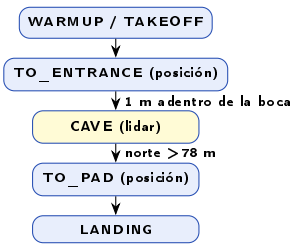
`steer()` piensa como el lidar (FLU: izquierda y giro a la izquierda positivos). El nodo ya convierte al mandar a PX4:

$$v_N = f\cos\psi + l\sin\psi$$
$$v_E = f\sin\psi - l\cos\psi$$
$$\dot{\psi}_{\text{NED}} = -\,\text{yawrate}$$

- f = adelante, l = izquierda, ψ = heading.
- Con $\psi = 0$ (al norte): izquierda = oeste $\Rightarrow v_E = -l$ ✓.

La altura va fija por posición ($z = -1.5$) y lo horizontal por velocidad: un **setpoint mixto**.

Flujo y calificación de la cueva



¿Por qué entrar por posición? Afuera, el rayo más largo no apunta a la boca: apunta al vacío de los lados.

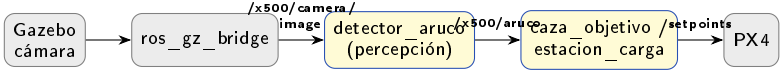
al aterrizar

```
cruzó la cueva: sí OK
del lidar a la roca, mínimo:
1.16 m (límite 0.6) OK
a 0.16 m de la plataforma OK
Misión cumplida
```

Contenido

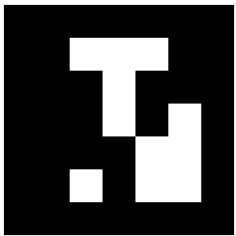
1. Panorama: qué es x500_retos
2. Preparar y correr
3. Marcos de referencia
4. px4_link.hpp: lo que comparten todas
5. Máquinas de estados
6. Cómo están hechos los mundos
7. Nivel 1: moverse bien
8. Nivel 2: decidir solo
9. Nivel 3: ver con cámara
10. Calificar, depurar y chuleta

Separar percepción de control



El detector **no sabe nada de PX4 ni de misiones**; las misiones **no saben nada de OpenCV**. Si mañana cambian la cámara, el detector o el marcador, las misiones no se tocan. Las misiones 6 y 7 usan el mismo detector.

ArUco en 1 lámina

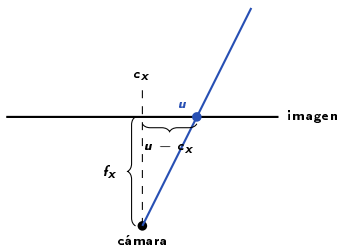


id 7, DICT_5X5_250

- Un cuadrado negro con una rejilla de 5×5 celdas: el patrón codifica un **número**.
- **Diccionario** DICT_5X5_250: 250 patrones (ids 0–249), los de TMR y de `tools/aruco`.
- Necesita un **margen blanco** alrededor (zona quieta): sin él, sobre fondo oscuro no se encuentra la orilla.
- El detector da el id y las **4 esquinas** en píxeles.

id	Dónde	Tamaño
7	techo de la camioneta sospechosa	1.2 m
3, 12, 25	coches estacionados (señuelos)	1.2 m
42	estación de carga	1.4 m

El modelo de cámara (pinhole)



`camera_info` trae la matriz K: f_x, f_y (focal en píxeles) y c_x, c_y (centro).

$$x_n = \frac{u - c_x}{f_x} \qquad y_n = \frac{v - c_y}{f_y}$$

x_n es la **tangente del ángulo** del rayo desde el centro. A una altura h:

$$\text{desplazamiento} \approx x_n \cdot h$$

Los mensajes propios

```
int32 id
float32 u           # [px] centro
float32 v           # [px]
float32 x_norm      # (u - cx) / fx
float32 y_norm      # (v - cy) / fy
float32 size_px     # [px] lado
```

Código 18: msg/Aruco.msg

```
std_msgs/Header header
uint32 width        # [px]
uint32 height
Aruco[] markers     # vacía si no ve nada
```

Código 19: msg/ArucoArray.msg

Se generan con `rosidl_generate_interfaces` en el `CMakeLists.txt` (como en la intro a ROS 2: interfaces y `rosidl`). En C++: `#include "x500_retos/msg/aruco_array.hpp"`.

detector_aruco: tu parte

```
cv_bridge::CvImageConstPtr gray = cv_bridge::toCvShare(msg, "mono8");  // ROS -> OpenCV, gris
const double fx = info_->k[0], cx = info_->k[2], fy = info_->k[4], cy = info_->k[5];
std::vector<int> ids;
std::vector<std::vector<cv::Point2f>> corners;
// TODO: detectar y llenar out.markers.
//   1. cv::aruco::detectMarkers(gray->image, dictionary_, corners, ids, params_);
//   2. Para cada uno: id, el centro (promedio de las 4 esquinas), x_norm, y_norm
//      y size_px (promedio de los 4 lados; cv::norm(a - b)).
```

Código 20: OpenCV 4.6 (el de Ubuntu 24.04): API cv::aruco::detectMarkers

Ojo

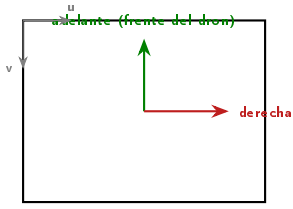
En OpenCV 4.7+ la API cambió a `cv::aruco::ArucoDetector`. Los tutoriales nuevos no compilan con el OpenCV de Ubuntu 24.04.

Ver lo que ve el detector

```
ros2 run rqt_image_view rqt_image_view /x500/aruco/debug
ros2 topic echo /x500/aruco --field markers
```

- `/x500/aruco/debug`: la imagen con los marcadores dibujados. Solo se publica si alguien la ve (dibujar cuesta).
- El detector procesa como mucho `max_rate` = 15 imágenes por segundo.
- Para probarlo sin volar: `tools/render_world.py` saca fotos de arriba de cualquier mundo; pásenle OpenCV en Python a esas fotos.

De un píxel a metros: la imagen

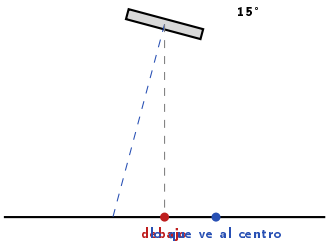


La cámara mira hacia abajo. **Arriba de la imagen = el frente del dron**, derecha = derecha. Con el dron nivelado y a h metros sobre el marcador:

$$\text{derecha} = x_n\, h \qquad \text{adelante} = -y_n\, h$$

(v crece hacia abajo de la imagen = hacia atrás del dron) y luego se rota por el heading a NED.

Pero la cámara no tiene gimbal



Para acelerar, el dron se inclina. La cámara se inclina con él:

$$12\ \text{m} \cdot \tan 15 \approx 3.2\ \text{m de error}$$

Persiguiendo una camioneta eso hacía brincar la posición estimada y el seguimiento se perdía.

Solución (ya hecha en `aruco_tools.hpp`): girar el rayo con la actitud **completa** del dron (cuaternión de PX4) y alargarlo hasta que baje h metros.

El rayo con la actitud

En el cuerpo (FRD) el rayo del marcador es $\mathrm{r}_b = (-y_n,\ x_n,\ 1)$. Con la actitud q:

$$r_{NED} = R(q)\,\mathrm{r}_b \qquad t = \frac{h}{r_{NED,z}} \qquad \Delta N = t\,r_{NED,x}, \quad \Delta E = t\,r_{NED,y}$$

```
const double height = -l.z - marker_h_;                // cámara sobre el marcador
const auto off = x500::marker_offset_ned(m, height, px4_.attitude().q);
const double n = l.x + off[0], e = l.y + off[1];       // el marcador en NED
```

Código 21: Así se usa (ya está en las misiones)

Si el rayo casi no baja (dron muy inclinado), la función regresa (0, 0): no hay que confiar en esa vista.

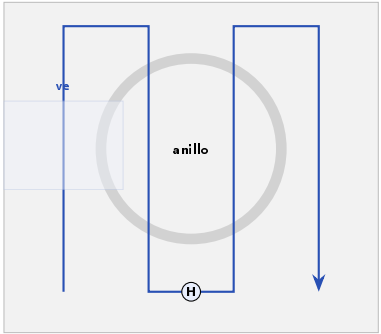
Misión 6: la caza del objetivo



Escenario: una camioneta sospechosa da vueltas en una zona de bodegas. Hay que **encontrarla**, **identificarla** por el ArUco 7 de su techo (no los señuelos), **dar su ubicación GPS** y **seguirla** desde arriba.

- Va a 3 m/s, una vuelta cada 55 s.
- **Lo nuevo**: perseguir algo que se **mueve**.

Buscar: el barrido “podadora”



A 15 m, con 100° de campo, la cámara ve

$$2\,h\tan(50) \approx 36 \text{ m de ancho}$$

Carriles norte–sur cada 25 m (se traslapan un poco) cubren todo el patio.

Entre carriles avanza a `search_speed` (5 m/s) con un punto que va 1 s adelante del dron y la velocidad como feedforward: si va muy rápido la cámara no alcanza a ver.

Señuelos: filtrar por id



Los coches estacionados también llevan ArUco (3, 12 y 25). El nodo los anuncia una vez y los ignora:

caza_objetivo

```
Veo el ArUco 3: es un señuelo, lo ignoro
Veo el ArUco 25: es un señuelo, lo ignoro
¡OBJETIVO ENCONTRADO! ArUco 7 en NED
norte 62.9, este 5.1 (lat ..., lon ...)
```

Seguir algo que se mueve



Si vas a donde la *viste*, cuando llegas ya no está ahí.
Hay que:

1. **Estimar su velocidad** (TODO 1).
2. **Predecir** y mandar **feedforward** (TODO 2):

$$p_{sp} = p_{vista} + v\,(edad + lead) \qquad v_{sp} = v$$

TODO 1: estimar la velocidad

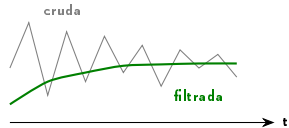
Con dos vistas de la camioneta, (p_1, t_1) y (p_2, t_2):

$$v_{\text{cruda}} = \frac{p_2 - p_1}{t_2 - t_1}$$

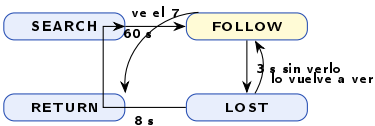
Cada vista trae ruido (píxeles, actitud): la velocidad cruda brinca. Se suaviza con un **filtro paso-bajas**:

$$v \leftarrow \alpha\, v_{\text{cruda}} + (1 - \alpha)\, v, \qquad \alpha \approx 0.3$$

- Solo si la vista anterior es **reciente** (<2 s).
- Y no **demasiado cercana** (>0.25 s): con Δt chico, el ruido domina.



Estados y calificación de la caza

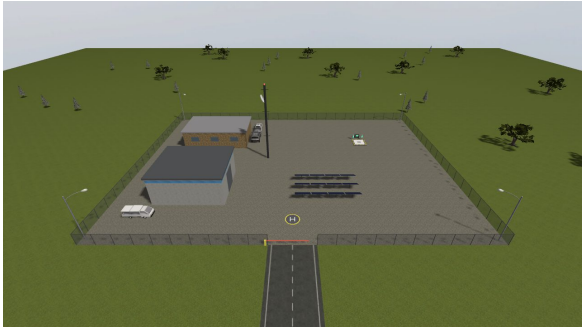


Durante los 60 s de seguimiento el nodo mide:

- **A la vista** $\geq 80\,\%$ del tiempo.
- **Distancia promedio** dron–camioneta (horizontal) < 1.5 m.

Sin los TODO, el dron va a donde la *vio*, se queda atrás y el promedio no pasa.

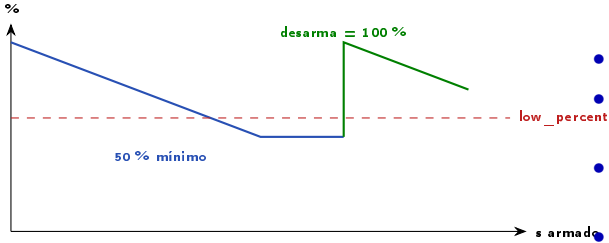
Misión 7: la estación de carga



Escenario: un dron de vigilancia patrulla la cerca de una base. Con batería baja, deja la patrulla, va a su estación, **aterriza con precisión** sobre el ArUco 42, "carga" y **regresa a patrullar justo donde se quedó**.

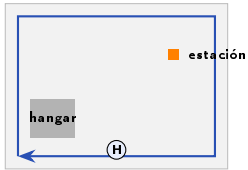
- **Viene de**: `battery_monitor` + `square_mission`.

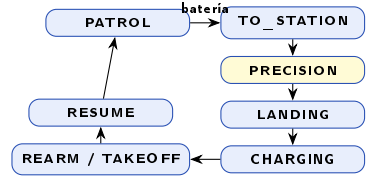
La batería en PX4 SITL



- Armado, baja de 100 % a 50 % en 30 s (`SIM_BAT_DRAIN` = 60 s) y **se queda en 50 %**.
- Al **desarmar** vuelve a 100 % al instante.
- O sea: aterrizar en la estación y desarmar **es cargar**.
- `low_percent` tiene que ser > 0.5 (si no, nunca se dispara).
- Se lee de `/fmu/out/battery_status_v1` (`remaining`, 0–1, a 1 Hz).

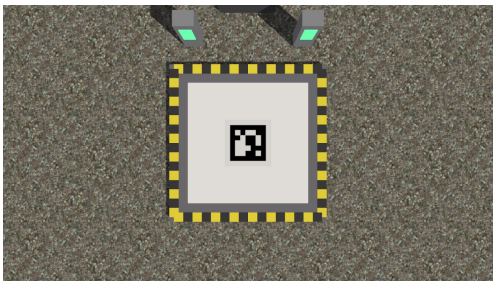
La patrulla y la máquina de estados





Al dejar la patrulla guarda **dónde estaba y en qué tramo**; al volver, va ahí primero. Patrulla a 8 m y a 4 m de la cerca: lejos del hangar (6.5 m de alto) y la comandancia.

El GPS te lleva cerca; la cámara te pone encima



La estación "según el mapa" (`station_north/east`) tiene **1.5 m de error a propósito**: la plataforma real está en N 30, E 18.

TODO: en cada ciclo de la bajada, un setpoint mixto:

- en x, y: **posición** = donde ve el ArUco;
- en z: **velocidad** = bajar a `descent_speed` solo si ya está **centrado** (<0.2 m); si no, 0 mientras se acomoda;
- si no lo ve: quieto y subir un poco para que vuelva a caber en la imagen.

Bajar centrado



- A ~0.8 m sobre la plataforma, PX4 termina con `land()`: detecta el contacto y desarma.
- **Éxito**: cada carga aterriza a **menos de 0.3 m** del centro del ArUco.
- Sin el TODO baja derecho sobre la coordenada del mapa y aterriza ~1.5 m chueco.

al terminar

```
carga 1: aterrizó a ... m del centro
(máximo 0.3) OK / FALLÓ
```

Contenido

1. Panorama: qué es x500_retos
2. Preparar y correr
3. Marcos de referencia
4. px4_link.hpp: lo que comparten todas
5. Máquinas de estados
6. Cómo están hechos los mundos
7. Nivel 1: moverse bien
8. Nivel 2: decidir solo
9. Nivel 3: ver con cámara
10. Calificar, depurar y chuleta

Criterio de éxito de cada misión

#	Misión	El nodo imprime “Misión cumplida” si…
1	Estrella	pasa a <0.5 m de cada vértice, regresa al centro y aterriza
2	Pilar	radio 3 ± 0.3 m, yaw $<10°$, $\geq 90\,\%$ de las vueltas y llega a 4 m
3	Entrega	secuencia completa sin tocar QGC y termina a <1.5 m de la H de la farmacia
4	Geocerca	el dron se detiene antes de pasar 1 m la cerca (prueben 2 y 6 m/s, varias direcciones)
5	Cueva	cruza, el lidar nunca a <0.6 m de la roca, aterriza a <1 m de la plataforma
6	Caza	a la vista $\geq 80\,\%$ de 60 s y a <1.5 m en promedio
7	Estación	cada carga aterriza a <0.3 m del centro del ArUco y termina la patrulla

La rúbrica

Criterio	Peso	Qué se revisa
Funciona	40 %	“Misión cumplida” en 3 corridas seguidas
Manejo de fallas	15 %	Aborta y regresa bien cuando se le provoca una falla
Código legible	20 %	Máquina de estados clara, comentarios como en `px4_examples`
Parámetros	10 %	Se ajusta con `-ros-args` sin recompilar
README + video	15 %	Otra persona lo corre solo con el README

Cada quien presenta su misión en 5 minutos frente al equipo. Explicar la máquina de estados en voz alta es donde se nota si de verdad la entendieron.

Provocar fallas a propósito

El 15 % de “manejo de fallas” se prueba así:

- **Matar el nodo** a media misión (Ctrl+C): en offboard PX4 debe entrar en failsafe; con comandos (misión 3) se queda en Hold.
- **Tomar el control desde QGC** (cambiar a Position o Hold): el nodo debe soltar y avisar “Fuera de OFFBOARD”.
- **Geocerca**: escaparse en distintas direcciones y velocidades con `vuelo_prueba`.
- **Cueva**: subir `max_speed` o bajar `stop_distance` hasta que falle; ¿cómo falla?
- **Estación**: subir `low_percent` a 0.9 para que vaya a cargar más seguido.

Problemas comunes (1/2)

Síntoma	Causa y arreglo
No arma; `No connection to the GCS`	QGC no está abierto: `vespo qgc` (o usar `vespo up -sim`)
`PX4 rechazó el comando 176`	El modo pedido no se puede ahora (p. ej. armar desde Land): pasar por Hold
El dron se va al suelo o hacia arriba	Signo de z: en NED arriba es negativo
La trayectoria se adelanta al dron	Tiempo con `now()` y la simulación lenta: usar `px4_.clock()`
"Fuera de OFFBOARD" sin tocar nada	El nodo dejó de publicar >0.5 s (un `return` antes de `offboard()`, un bloqueo)

Problemas comunes (2/2)

Síntoma	Causa y arreglo
`/x500/scan` o `/x500/camera/image` vacíos	El puente no está: usar el *launch*; revisar el modelo (`-custom`) y el nombre del mundo
El detector no ve nada	Falta `camera_info` (el detector la espera), marcador muy chico o sin margen blanco
La RAM de Gazebo crece sin parar	Cámara pesada o plugin de video de PX4: usar `x500_cam_abajo` y nuestros mundos
La laptop se apaga sola	Sin cargador: conectar
Un mundo modificado no cambia	Faltó `build_worlds.py` y `vespo build`

Chuleta

```
vespo build                                              # compilar
vespo up --sim --world <mundo> [--custom <modelo>]       # simular
ros2 run x500_retos <nodo> --ros-args -p <param>:=<valor>
ros2 launch x500_retos {cueva,caza,estacion}.launch.py
ros2 topic echo /fmu/out/vehicle_status_v1 --field nav_state
ros2 run rqt_image_view rqt_image_view /x500/aruco/debug
python3 src/x500_retos/tools/build_worlds.py             # regenerar mundos
gz topic -e -t /world/<mundo>/stats -n 1 | grep real_time    # ¿va a 1.0x?
```

Código 22: Lo de todos los días

Para cerrar

1. **Nivel 1**: muévanse bien (geometría, derivadas, comandos).
2. **Nivel 2**: que el dron decida (predecir, reaccionar a un sensor).
3. **Nivel 3**: que vea (percepción separada del control).

No se salta de nivel sin cerrar al menos dos misiones del anterior.



¿Preguntas?